Presentation Script

LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection

Slide 1 – Title 0:00–0:35

Good morning respected chairpersons, distinguished guests, and fellow researchers.

I am Debopam Sinha from the Department of Electronics and Communication Engineering, PES University, Bengaluru.

Today, I will present mywork titled *"LoRA-Based Resource-Efficient LLM Adaptation for RTL Hardware Trojan Detection."*

The central question behind this work is simple: can we use a modern large language model to detect and localize Hardware Trojans at RTL level, while keeping the computational requirement practical for an academic research environment?

Slide 2 – Presentation Outline 0:35–0:50

I will first briefly introduce the motivation and background.

Then I will explain the proposed workflow, dataset construction, training configuration, evaluation measures, and results.

Finally, I will discuss the limitations and future scope of this work.

Slide 3 – Introduction 0:50–1:35

Modern integrated circuits increasingly rely on third-party IP cores, hardware reuse, and complex design workflows.

This creates a serious security concern: a Hardware Trojan can be inserted into a design and remain inactive during normal verification. It may activate only under a rare trigger condition.

RTL-level analysis is valuable because it enables security assessment before synthesis and physical implementation.

However, conventional detection methods can become difficult to scale for complex or obfuscated RTL designs.

Recent LLM-based approaches are promising because Verilog is a structured language. But full model adaptation can require expensive GPUs and large computational resources.

mywork addresses this resource challenge using LoRA-based parameter-efficient fine-tuning.

Slide 4 – Background 1:35–2:10

A Hardware Trojan usually contains two important components.

The first is the **trigger**. This is a rare condition that activates the Trojan.

The second is the **payload**. This performs the malicious or unintended behavior after activation.

For example, a trigger may activate only when a specific counter value and input condition occur together. The payload may then alter a signal or modify the intended computation.

LLMs are relevant here because transformer models can learn relationships within serialized RTL code. This makes them useful candidates for RTL understanding, Trojan detection, and localization.

Slide 5 – Proposed Work 2:10–3:05

This slide shows the complete proposed workflow.

We begin with clean Verilog RTL samples. First, the RTL undergoes preprocessing, including formatting normalization, serialization, and lightweight structural obfuscation.

Next, controlled trigger and payload logic are inserted into selected RTL modules. During this stage, we record the inserted trigger and payload regions as ground-truth metadata.

Then, prompts are prepared for the model. Importantly, temporary Trojan markers, explicit annotations, and Trojan-specific comments are removed before final model use. This reduces the possibility of shortcut learning from obvious labels.

The base model is Qwen2.5-1.5B-Instruct. Instead of updating the entire model, we use LoRA adapters and keep most pretrained parameters frozen.

Finally, the adapted model performs two tasks: Trojan classification and localization of the trigger and payload regions.

Slide 6 – Dataset Description 3:05–3:45

mydataset was constructed from clean Verilog RTL samples obtained from the RTL-Coder repository.

We selected 300 clean RTL samples as the base set.

Using controlled Trojan insertion, we generated 300 Trojan-inserted RTL samples. Therefore, the final dataset is balanced, containing 600 samples in total: 300 clean and 300 Trojan samples.

We used a randomized stratified split:

- 420 samples for training,
- 90 samples for validation,
- 90 samples for testing.

Each split preserves an equal number of clean and Trojan samples.

Slide 7 – Trojan Injection Example 3:45–4:20

This slide illustrates a clean Verilog counter module and an example of controlled Trojan insertion.

The trigger condition activates when `en` is high and the counter reaches decimal value 10.

The payload signal becomes active after the trigger condition is satisfied.

This controlled design allows us to preserve valid RTL behavior while creating a known Trojan region for evaluation.

The purpose is not to claim that this represents every real-world Trojan. It provides a reproducible controlled environment for evaluating detection and localization.

Slide 8 – Metadata and Prompt Sanitization 4:20–4:55

Here, the inserted trigger and payload logic are shown together with the metadata recorded during generation.

The `trigger_nodes` and `payload_nodes` metadata identify the inserted RTL regions and provide ground truth for localization evaluation.

A key point is that markers such as `HT_START`, `HT_END`, and Trojan-specific comments are generation-time annotations only.

They are removed during final prompt preparation. Therefore, the model does not receive explicit Trojan markers during inference.

Slide 9 – Training Configuration 4:55–5:35

For model adaptation, we used Qwen2.5-1.5B-Instruct with LoRA-based PEFT.

The LoRA rank is 8. We used FP16 precision, a learning rate of 2×10^{-4}, batch size 1, gradient accumulation of 8, and trained for 3 epochs.

Only 2.17 million parameters were trainable. This is approximately 0.141 percent of the full model parameters.

The implementation ran on Google Colab using an NVIDIA Tesla T4 GPU, with a training runtime of approximately six minutes.

This configuration demonstrates that the workflow can be reproduced without dedicated high-end computing infrastructure.

Slide 10 – Performance Measures 5:35–6:15

We evaluate binary Trojan detection using fmystandard metrics: accuracy, precision, recall, and F1-score.

Accuracy measures overall classification correctness.

Precision indicates how reliable a Trojan prediction is.

Recall measures how many actual Trojan samples are detected.

The F1-score balances precision and recall.

For localization, we use Trigger Localization Coverage, or TLC, and Payload Localization Coverage, or PLC.

TLC measures the fraction of ground-truth trigger nodes covered by predicted trigger nodes. PLC uses the same principle for payload nodes.

Slide 11 – Results and Analysis 6:15–7:25

On the held-out test set of 90 samples, the framework achieved 98.89 percent accuracy and 98.88 percent F1-score.

Precision was 100 percent, which means that no clean sample was incorrectly predicted as Trojan in this test set.

Recall was 97.78 percent. One Trojan sample was predicted as clean.

The confusion matrix shows that all 45 clean samples were correctly classified. Among the 45 Trojan samples, 44 were correctly detected and one was missed.

For localization, both TLC and PLC reached 98.89 percent under this controlled evaluation.

From the resource perspective, the method used LoRA-based PEFT, 2.17 million trainable parameters, a Tesla T4 GPU, and approximately six minutes of runtime.

These results should be interpreted within the controlled synthetic RTL environment used in this study.

Slide 12 – Conclusion and Future Scope 7:25–8:25

To conclude, we developed a resource-efficient LoRA-based framework for RTL Hardware Trojan detection and localization.

The work combines controlled Trojan generation, prompt sanitization, parameter-efficient LLM adaptation, and localization-aware evaluation.

On the controlled held-out test set, the framework achieved 98.89 percent accuracy, 98.88 percent F1-score, and 98.89 percent TLC and PLC.

The main contribution is not only the detection performance. It is also the ability to perform this experiment with a lightweight adaptation strategy on a Tesla T4 GPU.

At the same time, we acknowledge the current limitations.

The dataset is synthetic, the analysis is limited to RTL level, and the randomized split does not explicitly enforce template-level separation.

Future work will evaluate larger and more heterogeneous benchmarks, real-world Trojan datasets, stronger generalization protocols, and gate-level or post-synthesis analysis.

Slide 13 – References 8:25–8:40

These are the key references that informed this work, including PEARL, TrojanWhisper, LoRA, RTL-Coder, and foundational Hardware Trojan literature.

Slide 14 – Thank You 8:40–8:50

Thank you for ymyattention.

I will be happy to answer any questions.